

TEAM – 2

KIDUS MIKAEL BIRHANU

SOHAN ARUN

SREEDHARANI MACHUNURU

1. Introduction

This lab focuses on building foundational skills in Amazon EC2 and related AWS services, including VPC configurations, monitoring, and network communication. The tasks involve setting up a VPC with public and private subnets, launching and configuring EC2 instances, enabling CloudWatch monitoring for performance metrics, and testing network communication between instances. By deploying resources, generating workloads, and analyzing performance, this experience provides hands-on practice with cloud infrastructure management and monitoring.

2. Task 1 – Environment Setup & VPC Configuration

2.1 VPC Configuration

In this lab, we successfully set up a Virtual Private Cloud (VPC) to simulate a production-like environment with both public and private subnets. Below are the steps and key configurations carried out during the process and motivation behind selection:

1. **IPv4 CIDR block: 10.0.0.0/16**

We selected this because it provides a large address space (65,536 IPs) for our VPC, ensuring ample room for scalability and resource allocation in the future.

2. **Number of Availability Zones: 1**

Limiting the configuration to a single Availability Zone reduces complexity and cost. This is suitable for the lab environment, which does not require high availability or fault tolerance across multiple zones.

3. **Number of Public Subnets: 1**

A single public subnet fulfils the lab's requirements while keeping the design straightforward and cost-effective. Public subnets are typically used for resources like bastion hosts or load balancers that need internet access.

4. **Number of Private Subnets: 1**

One private subnet meets the lab's requirements for isolating resources such as databases or application servers that do not need direct internet access.

5. **Public Subnet CIDR block: 10.0.0.0/24**

This allocation reserves 256 IPs for resources requiring public internet access. It balances the needs of the lab while conserving the overall address space.

6. **Private Subnet CIDR block: 10.0.1.0/24**

This allocation reserves 256 IPs for private resources, such as backend systems, ensuring sufficient IP availability without over-allocating.

7. NAT Gateways: In 1 AZ

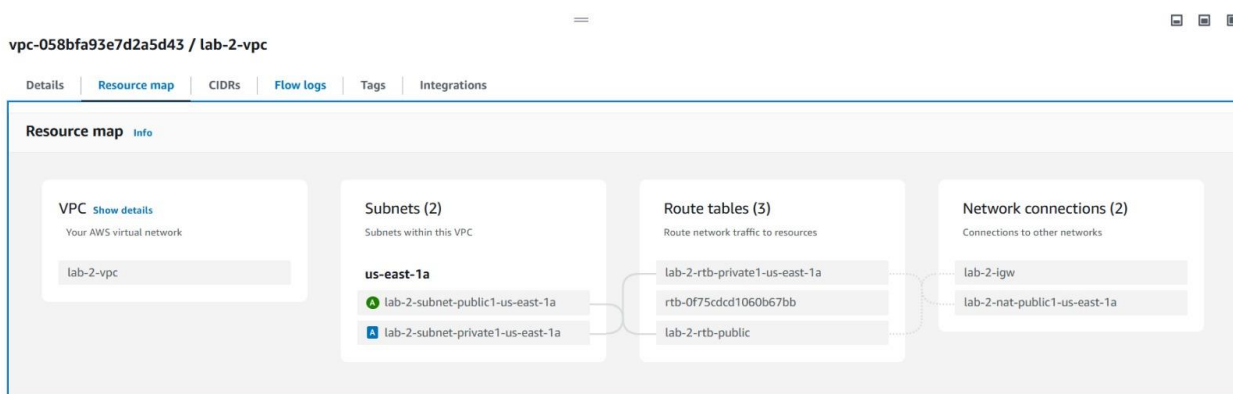
A NAT Gateway is essential for allowing resources in the private subnet to access the internet for updates or external API calls while maintaining their isolation. Placing it in a single AZ aligns with the cost and complexity reduction goals.

8. VPC Endpoints: None

Since the lab does not require private connectivity to AWS services, VPC endpoints were not necessary. This simplifies the setup and reduces costs.

9. DNS Hostnames and DNS Resolution: Enabled

Enabling these features ensures that resources within the VPC can resolve domain names, supporting seamless communication with internet-facing services.



This configuration allowed us to successfully create a **VPC** with an **Internet Gateway**, a **public subnet**, a **private subnet** within a single Availability Zone, along with **two route tables** and a **NAT Gateway**. These choices were motivated by the lab's requirements for simplicity, cost-efficiency, and clear separation of public and private resources.

2.2 Security Group Configuration

In this section, we created a security group associated it with the VPC configured earlier. We added an inbound rule to allow SSH access exclusively from my IP, ensuring secure and restricted remote connectivity to the EC2 instance which we will create in the following steps. The following are steps we followed with motivation behind selection of configurations.

1. Type: SSH

The **SSH** type allows secure access to the EC2 instance using **port 22**. SSH is a widely used protocol for remote administration and secure communication with Linux-based servers. This ensures we can manage and troubleshoot the instance securely.

2. Source: My IP (203.191.137.10/32)

- **Reason:** The source is limited to **My IP**, which means only the user's specific public IP address can initiate an SSH connection. This significantly enhances security by preventing access from other IPs, minimizing exposure to unauthorized access or brute-force attacks.

Inbound rules [Info](#)

Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info
sgr-0a196eef3ab166b	SSH	TCP	22	M... ▲ 1.191.137.10/32 × - Custom - Anywhere-IPv4 - Anywhere-IPv6 - My IP ☒	

[Add rule](#) [Delete](#)

By configuring the inbound security rule to allow SSH access (Port 22) exclusively from the current IP address (My IP), we implemented the principle of least privilege. This setup ensures that only authorized users with the specific IP can access the EC2 instance, reducing the risk of unauthorized access. Such a targeted rule balances security and functionality, providing safe and secure management access to the instance.

2.3 EC2 Instance Setup

In this section, we launched an EC2 instance with the necessary configurations to support secure and efficient operations. Below are the steps we followed, along with the rationale behind each selection:

1. **Amazon Machine Image (AMI):**
 - **Selected:** "Ubuntu Server 24.04 LTS (HVM), SSD Volume Type"
 - We selected this because in the lab we were told it is preferred to have ubuntu machine
2. **Instance Type:**
 - **Selected:** t2.micro
 - We selected this because The t2.micro instance is a cost-effective option with sufficient resources
3. **Key Pair:**
 - **Action:** Created a new key pair named "lab-2.pem" for secure access. And was downloaded immediately.
4. **Network Settings:**
 - **VPC Association:** Associated the instance with the VPC configured earlier.
 - This ensures the instance operates within the defined private and public networking boundaries.
 - **Subnet Selection:** Attached the instance to the **public subnet**.
 - Resources in the public subnet can be accessed directly from the internet when necessary.
 - **Auto-Assign Public IP:** Enabled.
 - Assigning a public IP allows remote access to the instance over the internet.
 - **Security Group:** Attached the previously configured security group.

- The security group restricts access to SSH from only the specified IP address, ensuring secure and controlled connectivity.

5. Connecting to the Instance:

- **Command Used:** `ssh -i "lab-2.pem" ubuntu@ec2-44-192-88-113.compute-1.amazonaws.com`
- The ssh command, combined with the private key, provides a secure method to log in to the instance. Ensuring the command is run from the same directory as the key pair file avoids permission issues.

6. Instance Setup:

- **Updated Package List:** `sudo apt-get update`
 - Updating the package list ensures the latest versions of software and dependencies are available for installation.
- **Installed Required Software:** `sudo apt-get install sysbench apache2 apache2-utils`
 - **Reason:**
 - **Sysbench:** Used for benchmarking and performance testing.
 - **Apache Web Server:** Hosts web applications and serves web content.
 - **Apachebench:** Provides tools for testing the performance of web servers

2.4 AMI Creation

We navigated to the Instances console, selected our instance, and went to Actions > Image and Templates > Create Image. This allowed us to capture an AMI (Amazon Machine Image), which has **Sysbench, Apache Web Server and Apachebench**. which we later used to launch the private EC2 instance.

3. Task 2 – Monitoring

3.1 Enabling Instance Monitoring

In the Monitoring section, we navigated to the Instances console, selected our instance, and followed this path: Actions > Monitor and troubleshoot > Manage detailed monitoring. From there, we enabled Detailed Monitoring for our instance.

3.2 Performance metrics

Next, we accessed CloudWatch and created a custom dashboard to monitor key performance metrics. The selected metrics included CPUUtilization, EBSReadOps, EBSWriteOps, EBSReadBytes, and EBSWriteBytes.

Motivation behind the selected metrics:

1. CPUUtilization (Blue Line):

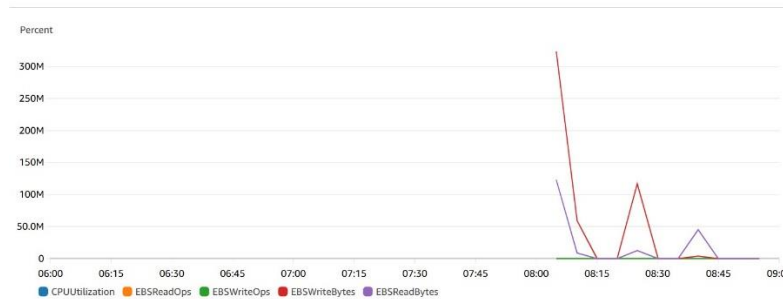
Monitoring CPU usage is critical to understanding the workload on the instance. High CPU utilization could indicate resource bottlenecks, inefficient processes, or increased demand that may require scaling or optimization. It ensures we can respond proactively to performance issues.

2. EBSReadOps (Orange Line) and EBSWriteOps (Green Line):

These metrics track the number of read and write operations on the EBS volumes. They are essential for analyzing the I/O workload. Monitoring these ensures we can identify potential issues like slow read/write performance due to high operations or inefficient data access patterns.

3. EBSWriteBytes (Red Line) and EBSReadBytes (Purple Line):

These metrics measure the amount of data written to and read from the EBS volumes. By tracking these metrics, we can observe spikes in data throughput that might impact application performance. This helps ensure the storage is performing optimally under varying workloads.



The dashboard above was created to provide a comprehensive view of the instance's performance by focusing on key metrics:

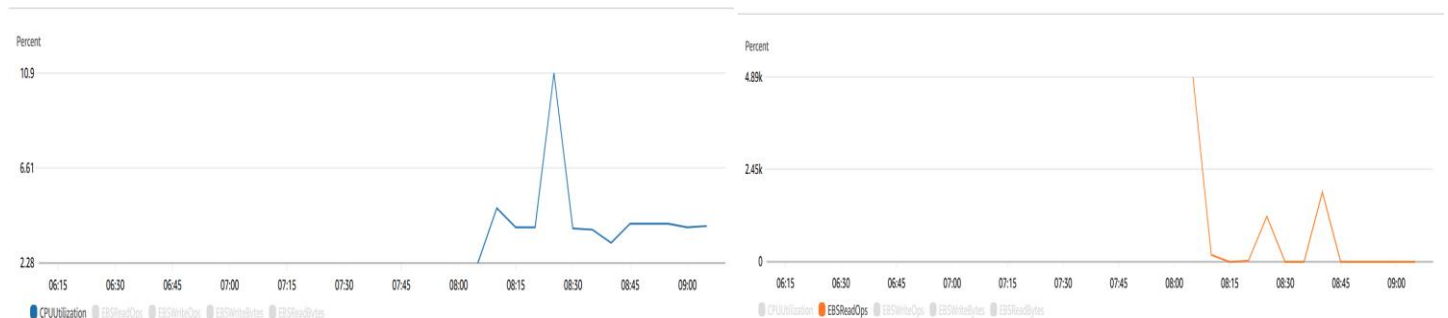
CPU Utilization – to monitor computing performance.

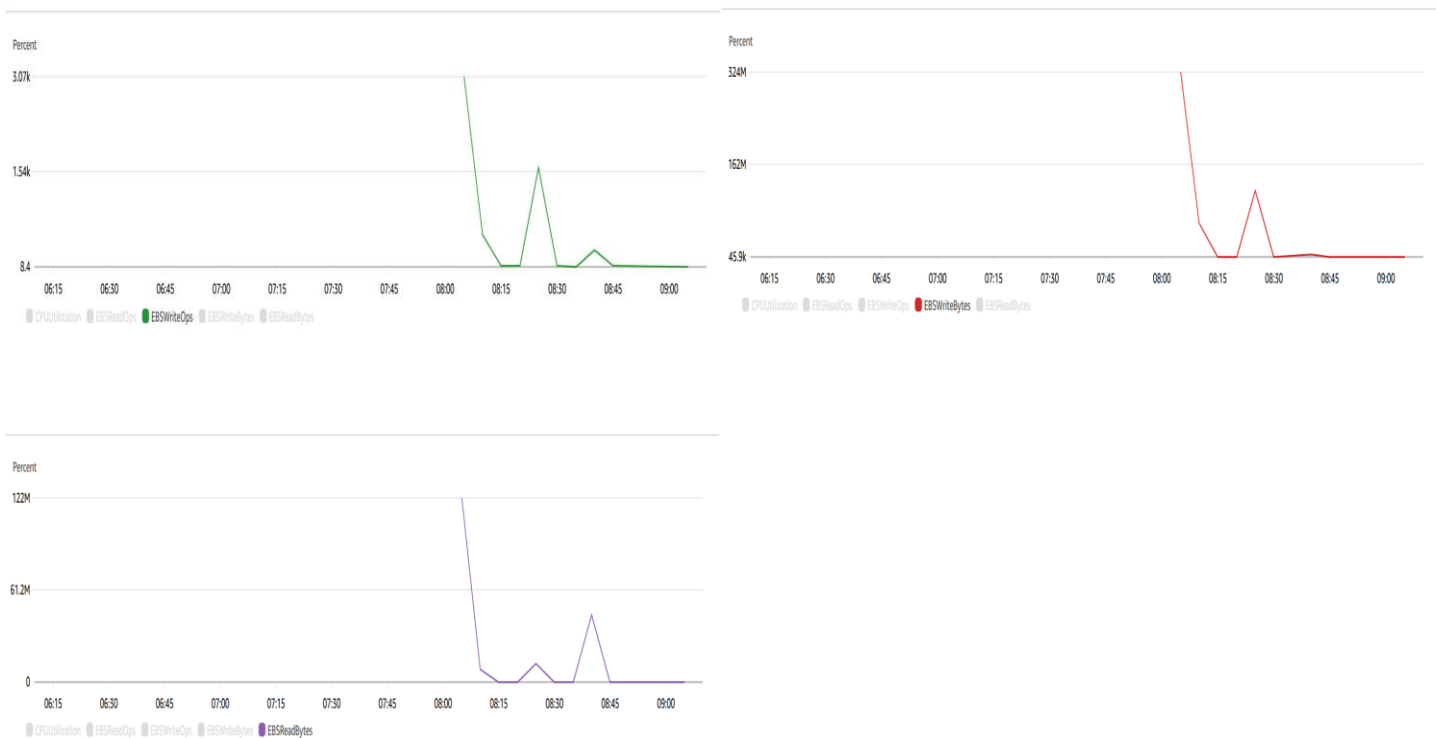
EBS Read/Write Operations and Throughput – to evaluate storage performance.

By correlating these metrics, we can effectively identify performance bottlenecks, optimize resource utilization, and ensure the system is capable of handling current and future workloads. This data-driven approach enables proactive scaling, efficient troubleshooting, and performance optimization.

3.3 Generating Workloads

Before running the sysbench tests, the performance metrics for the EC2 instance, such as CPU utilization and disk I/O, showed minimal activity. The CPU utilization was near idle, reflecting low computational workload, while the EBS metrics (like read/write operations and throughput) recorded very low values, indicating minimal disk activity. These baseline measurements provided a clear starting point, allowing us to observe significant variations and performance impacts once the CPU stress tests and disk I/O workloads were executed.





We executed a series of **sysbench** commands to generate CPU and disk-intensive workloads on the EC2 instance for performance testing and monitoring:

1. CPU Stress Test:

for each in 1 2 4 8 16 32 64; do sysbench cpu --cpu-max-prime=200000 --threads=\$each run; done

This command ran a CPU stress test with increasing thread counts (1 to 64). It measured the instance's ability to handle intensive computational tasks, simulating high CPU utilization.

```
ubuntu@ip-10-0-0-30:~$ for each in 1 2 4 8 16 32 64; do sysbench cpu --cpu-max-prime=200000 --threads=$each run; done
sysbench 1.0.20 (using system LuaJIT 2.1.0-beta3)

Running the test with following options:
Number of threads: 1
Initializing random number generator from current time

Prime numbers limit: 200000

Initializing worker threads...

Threads started!
```

```

CPU speed:
  events per second:    14.43

General statistics:
  total time:           10.0469s
  total number of events: 145

Latency (ms):
  min:                  68.20
  avg:                  69.28
  max:                  88.53
  95th percentile:     70.55
  sum:                  10045.22

Threads fairness:
  events (avg/stddev):  145.0000/0.00
  execution time (avg/stddev): 10.0452/0.00

sysbench 1.0.20 (using system LuaJIT 2.1.0-beta3)

Running the test with following options:
Number of threads: 2
Initializing random number generator from current time

Prime numbers limit: 200000

```

```

Prime numbers limit: 200000

Initializing worker threads...

Threads started!

CPU speed:
  events per second:    14.39

General statistics:
  total time:           13.3436s
  total number of events: 192

Latency (ms):
  min:                  3608.49
  avg:                  4249.61
  max:                  4520.44
  95th percentile:     4437.27
  sum:                  815924.75

Threads fairness:
  events (avg/stddev):  3.0000/0.00
  execution time (avg/stddev): 12.7488/0.12

ubuntu@ip-10-0-0-30:~$ |

```

2. Disk I/O Test - File Preparation:

sysbench --test=fileio --file-total-size=8G --file-num=3 prepare

This step prepared the disk for testing by creating a total of **8GB** of test files, split into **3 files**. It ensured enough data was available to generate a meaningful disk I/O workload.

```

ubuntu@ip-10-0-0-30:~$ sysbench --test=fileio --file-total-size=8G --file-num=3 prepare
WARNING: the --test option is deprecated. You can pass a script name or path on the command line without any options.
sysbench 1.0.20 (using system LuaJIT 2.1.0-beta3)

3 files, 2796202Kb each, 8191Mb total
Creating files for the test...
Extra file open flags: (none)
Creating file test_file.0
Creating file test_file.1
Creating file test_file.2
8589950976 bytes written in 133.24 seconds (61.48 MiB/sec).
ubuntu@ip-10-0-0-30:~$ ls
test_file.0 test_file.1 test_file.2
ubuntu@ip-10-0-0-30:~$ |

```

3. Disk I/O Stress Test - Random Read/Write:

for each in 1 4 8 16 32; do sysbench fileio --file-total-size=8G --file-test-mode=rndwr --time=240 --max-requests=0 --file-block-size=4K --file-num=3 --threads=\$each run; done

This command performed a random read/write stress test over 240 seconds with increasing thread counts (1 to 32). It measured disk performance under concurrent read/write operations using 4KB blocks.

```
ubuntu@ip-10-0-0-30:~$ for each in 1 4 8 16 32; do sysbench fileio --file-total-size=8G --file-test-mode=rndwr --time=240 --max-requests=0 --file-block-size=4K --file-num=3 --threads=$each run; done
sysbench 1.0.20 (using system LuaJIT 2.1.0-beta3)

Running the test with following options:
Number of threads: 1
Initializing random number generator from current time

Extra file open flags: (none)
3 files, 2.6667GiB each
8GiB total file size
Block size 4KiB
Number of IO requests: 0
Read/Write ratio for combined random IO test: 1.50
Periodic FSYNC enabled, calling fsync() each 100 requests.
Calling fsync() at the end of test, Enabled.
Using synchronous I/O mode
Doing random write test
Initializing worker threads...

Threads started!
```

```
File operations:
  reads/s:                0.00
  writes/s:               1043.25
  fsyncs/s:               31.30

Throughput:
  read, MiB/s:            0.00
  written, MiB/s:         4.08

General statistics:
  total time:              240.0086s
  total number of events:  257901

Latency (ms):
  min:                     0.00
  avg:                     0.93
  max:                     146.00
  95th percentile:        2.52
  sum:                     239447.96

Threads fairness:
  events (avg/stddev):     257901.0000/0.00
  execution time (avg/stddev): 239.4480/0.00

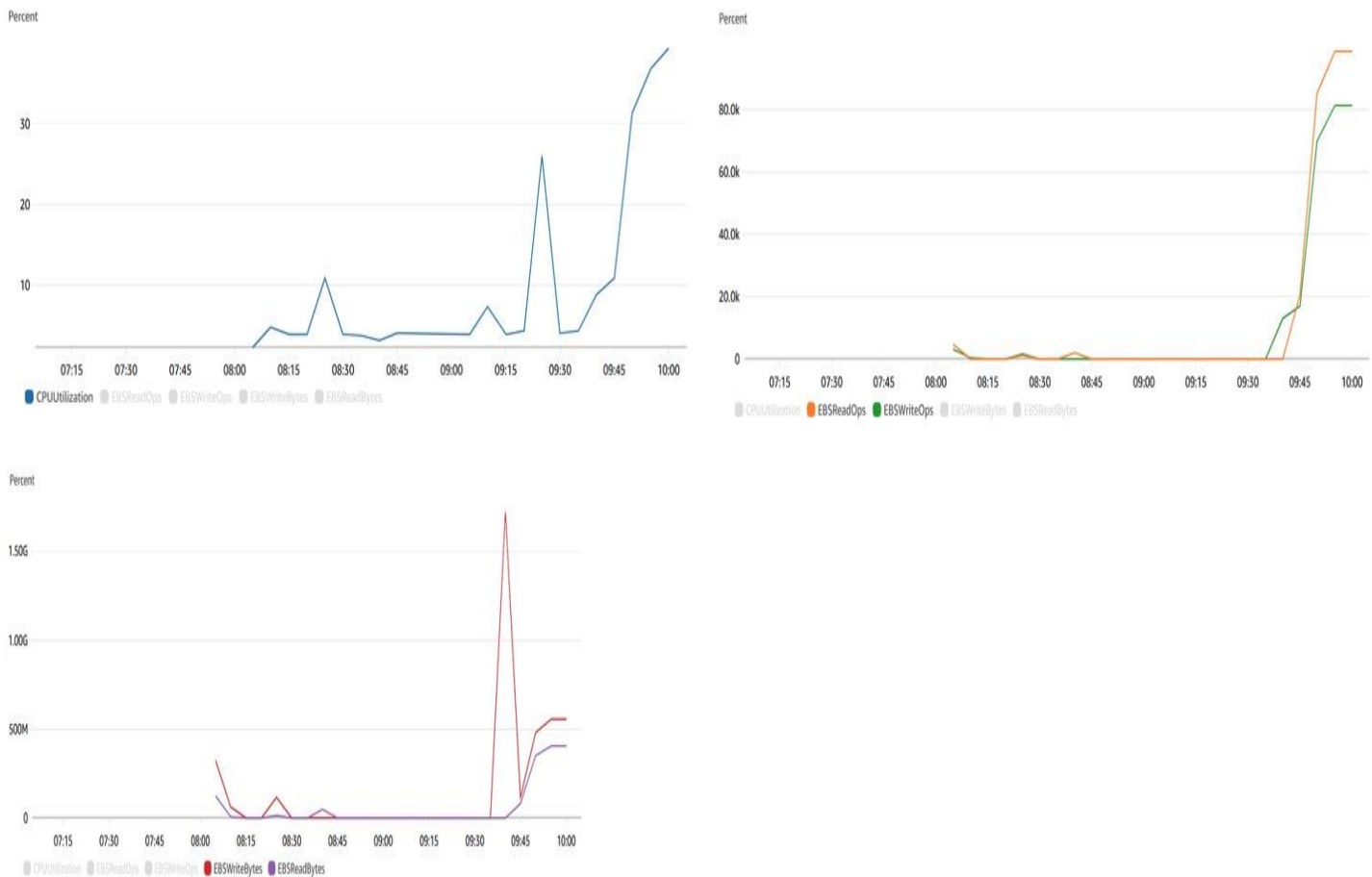
sysbench 1.0.20 (using system LuaJIT 2.1.0-beta3)

Running the test with following options:
Number of threads: 4
Initializing random number generator from current time
```

```
Extra file open flags: (none)
3 files, 2.6667GiB each
8GiB total file size
Block size 4KiB
Number of IO requests: 0
```

These commands allowed us to simulate and observe high CPU and disk loads, providing insights into the performance capabilities of the EC2 instance and its storage system.

After running the sysbench tests, the performance metrics showed significant increases. CPU utilization spiked as thread counts scaled from 1 to 64, reflecting the EC2 instance's ability to handle intensive computational tasks. Similarly, EBS metrics such as DiskReadOps, DiskWriteOps, and throughput (read/write bytes) increased during the disk I/O tests, demonstrating the impact of concurrent file operations on storage performance. These results validated the monitoring setup, clearly showing resource utilization under high workloads.



4. Task 3 – Communication

4.1 WebServer Setup

In this section, we set up an EC2 instance named "**webserver**" to host a web server. Below are the configurations and motivations behind each step:

1) Instance Creation:

- **Name:** Assigned the instance the name "**webserver**" for clear identification.
- **AMI:** Selected the "**lab-2**" AMI, which is an image of the previously created public instance.
 - **Reason:** The AMI already includes pre-installed software such as Sysbench, Apache Web Server, and Apachebench, ensuring consistency and reducing setup time.

2) Instance Type:

- **Selected:** t2.micro
 - The t2.micro instance type is cost-effective and sufficient for hosting lightweight web applications in a lab environment.

3) **Key Pair:**

- **Action:** Used the same key pair as the public instance.
 - Reusing the key pair simplifies management while maintaining secure access.

4) **Network Settings:**

- **VPC Association:** Associated the instance with the **VPC** created earlier.
 - Ensures the instance operates within the defined network configuration.
- **Subnet Selection:** Selected the **private subnet**.
 - Placing the web server in a private subnet enhances security by restricting direct access to the internet.
- **Auto-Assign Public IP:** Disabled.
 - Prevents public access to the instance, ensuring it remains accessible only from within the VPC.
- **Security Group:** Attached the previously configured **security group**.
 - Maintains secure and controlled access policies.

4.2 HTTP Communication Between Instances

In this section, we verified HTTP communication between the public instance and the web server in the private subnet.

1) **Connecting to the Public Instance:**

- **Action:** Used SSH to log into the public instance.
- The public instance serves as a gateway to interact with the private subnet resources.

2) **HTTP Request:**

- **Command Used:** curl <http://10.0.1.115/index.html>
- This command sends an HTTP request to the web server hosted in the private subnet to fetch the index.html page.

3) **Response:**

- **Output:** Successfully retrieved HTML and CSS content.
- **Reason:** This confirms that the web server in the private subnet is reachable via HTTP requests from the public instance, demonstrating proper network configuration and communication.

4.3 Stress Testing

To evaluate the performance of the web server, we performed stress testing using the **ApacheBench** (ab) tool. Below are the details of the test:

- **Command Executed:** ab -n 200000 -c 20 <http://ip-10.0.1.115/index.html>

- **Explanation of the Command:**
 - **ab:** ApacheBench, a tool used to test and benchmark the performance of HTTP servers.
 - **-n 200000:** Specifies the total number of HTTP requests to be sent to the server.
 - A high number of requests helps evaluate the server's ability to handle a heavy load.
 - **-c 20:** Specifies the number of concurrent requests to be sent at a time.
 - Simulating multiple simultaneous users accessing the server tests its concurrency handling capabilities.
 - **http://ip-10.0.1.115/index.html:** The URL of the resource being tested.
 - This ensures the requests target the index.html page hosted on the web server in the private subnet.
- **Purpose:** To assess the web server's response time, throughput, and ability to handle a high volume of concurrent requests and To identify potential bottlenecks or areas for performance improvement.

4.4 stress test monitoring

To monitor the performance of the web server during the stress test, we created a dashboard and selected specific metrics to analyze its behavior. Below are the metrics chosen and the rationale for their selection:

1. Selected Metrics:

- **NetworkPacketsIn:**
 - Tracks the number of packets received by the instance. During a stress test, this metric helps identify the volume of incoming HTTP requests and ensures the server is correctly receiving the traffic load.
- **NetworkPacketsOut:**
 - Monitors the number of packets sent by the instance. This metric indicates the server's ability to process requests and send responses efficiently, which is critical for ensuring users receive data without delays.
- **NetworkIn:**
 - Measures the total volume of data (in bytes) received by the instance. This metric helps evaluate whether the server can handle the data load generated by the high volume of HTTP requests during the stress test.
- **NetworkOut:**
 - Tracks the total volume of data (in bytes) sent by the instance. Monitoring this metric ensures the server can handle and respond to outgoing data demands, such as serving the requested HTML and CSS files to clients.

2. Monitoring Metrics

For the two instances (public and private), we tracked a total of **8 metrics** using the dashboard, providing a holistic view of network performance:

- **Before Stress Command:** The following screenshots capture the state of the dashboard before executing the stress test commands. These provide the baseline network activity, indicating minimal traffic as the instances were idle or under normal operation.



Executing Stress Test: Screenshots were taken during and after the execution of the stress test to monitor its impact

```
ubuntu@ip-10-0-0-147:~$ ab -n 100000 -c 10 http://10.0.1.115/index.html
This is ApacheBench, Version 2.3 <$Revision: 1879490 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 10.0.1.115 (be patient)
Completed 10000 requests
Completed 20000 requests
Completed 30000 requests
Completed 40000 requests
Completed 50000 requests
Completed 60000 requests
Completed 70000 requests
Completed 80000 requests
Completed 90000 requests
Completed 100000 requests
Finished 100000 requests

Server Software:      Apache/2.4.58
Server Hostname:      10.0.1.115
Server Port:          80

Document Path:        /index.html
Document Length:       10671 bytes

Concurrency Level:     10
Time taken for tests:   34.128 seconds
Complete requests:      100000
Failed requests:         0
Total transferred:      1094500000 bytes
HTML transferred:       1067100000 bytes
Requests per second:    2930.17 [#/sec] (mean)
Time per request:       3.413 [ms] (mean)
Time per request:       0.341 [ms] (mean, across all concurrent requests)
Transfer rate:          31319.04 [Kbytes/sec] received

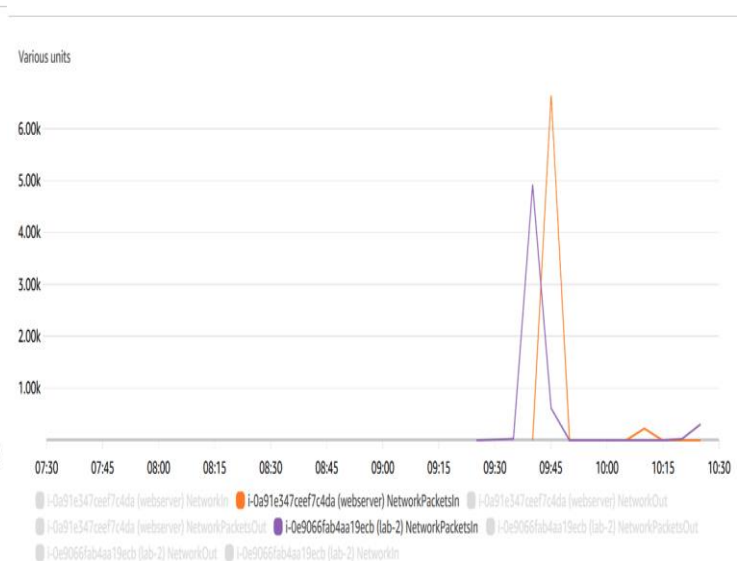
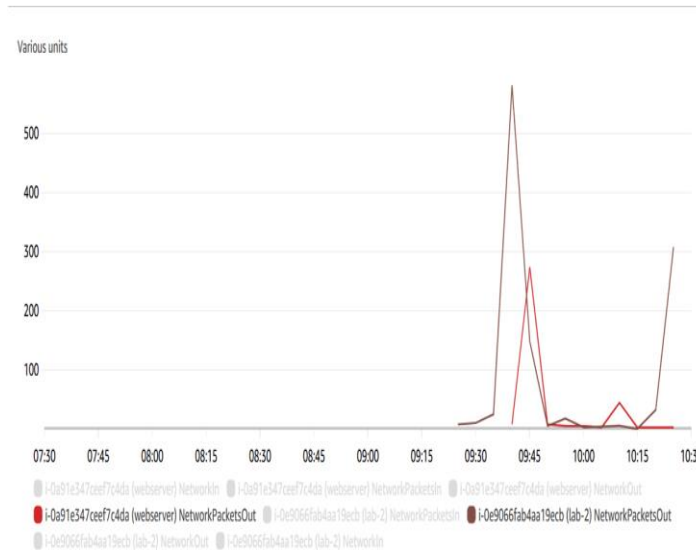
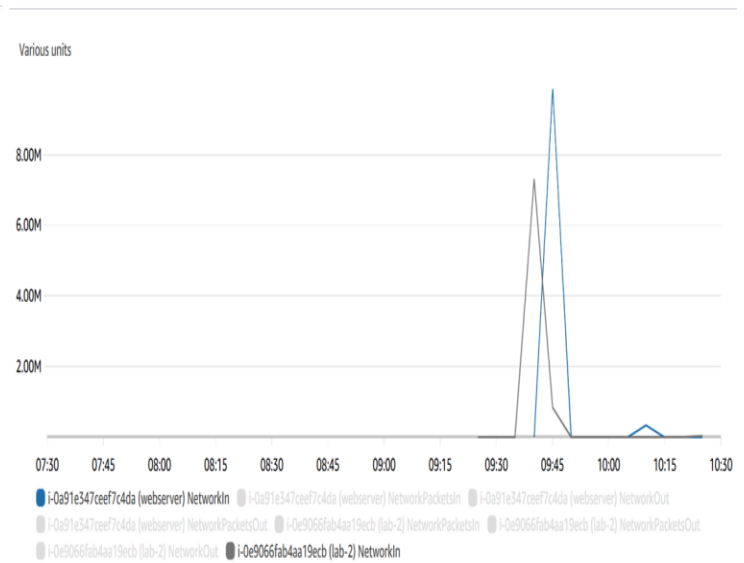
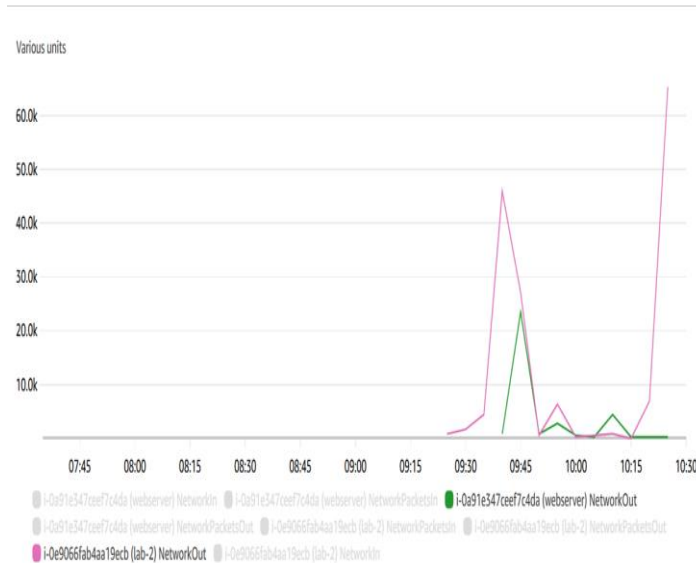
Connection Times (ms)
  min  mean[+/-sd] median  max
Connect:  0    1   0.8      1   10
Processing:  0    2   1.3      2   18
Waiting:  0    2   1.0      2   16
Total:  0    3   1.5      4   18

Percentage of the requests served within a certain time (ms)
 50%    4
 66%    4
 75%    4
 80%    4
 90%    5
 95%    6
 98%    7
 99%    9
100%   18 (longest request)
ubuntu@ip-10-0-0-147:~$
```

After Stress Command: The following screenshots show the network activity after the stress test.

Observations:

- **Increased Metrics:** Significant spikes in NetworkPacketsIn, NetworkPacketsOut, NetworkIn, and NetworkOut were observed, reflecting the high volume of HTTP requests and responses during the test.
- **Public Instance:** Facilitated the traffic for HTTP requests originating from the stress test tool.
- **Private Instance:** Handled the bulk of the load, serving the requested index.html content efficiently.



5. Discussion

The lab exercise involved setting up a VPC with public and private subnets, configuring security groups, launching EC2 instances, and monitoring performance under simulated workloads. The VPC configuration ensured resource isolation and secure internet access for private instances via a NAT Gateway. SSH access was restricted to a specific IP for secure management.

We launched an EC2 instance, installed sysbench, Apache Web Server, and Apachebench, and conducted CPU and disk I/O stress tests. Metrics like CPU utilization and disk I/O spiked during testing, demonstrating the instance's ability to handle high workloads. A second EC2 instance was launched in the private subnet to test HTTP communication, with stress tests validating network configuration and performance monitoring via CloudWatch.

An issue with an incorrect SSH command underscored the importance of precision in configurations. This hands-on experience enhanced our understanding of AWS resource management, workload testing, and the critical role of attention to detail in cloud infrastructure setup.

6. Conclusion

This lab provided hands-on experience in configuring and managing AWS resources, including VPCs, EC2 instances, and performance monitoring tools. By setting up secure, isolated environments and testing resource performance under simulated workloads, we gained practical insights into cloud infrastructure management. The challenges encountered, such as resolving an SSH command issue, emphasized the importance of precision in configurations. Overall, the exercises enhanced our understanding of AWS resource optimization, network communication, and system performance analysis.